

1. Project Title

MyHR Mobile Application

2. Project Description

Developed for the Mobile and Ubiquitous Computing course, this Flutter-based mobile app demonstrates how key Flutter concepts—such as user interface design, navigation, form handling, and database integration—are implemented.

The system includes the Main, Home, Attendance, Leave, Announcement, Profile, and Entry approval pages. Each page has its own purpose and function in the myHR app. The software also keeps data locally using SQLite (sqflite) so that users may execute simple operations, including adding and seeing records.

Generally, this project, with correct UI design and data management approaches, helps one to learn how to create a structured and working mobile application utilising Flutter.

3. Project Objectives

- To design a user-friendly mobile application using Flutter
- To implement navigation using BottomNavigationBar and Drawer
- To apply form validation and user input handling
- To integrate SQLite database for data storage

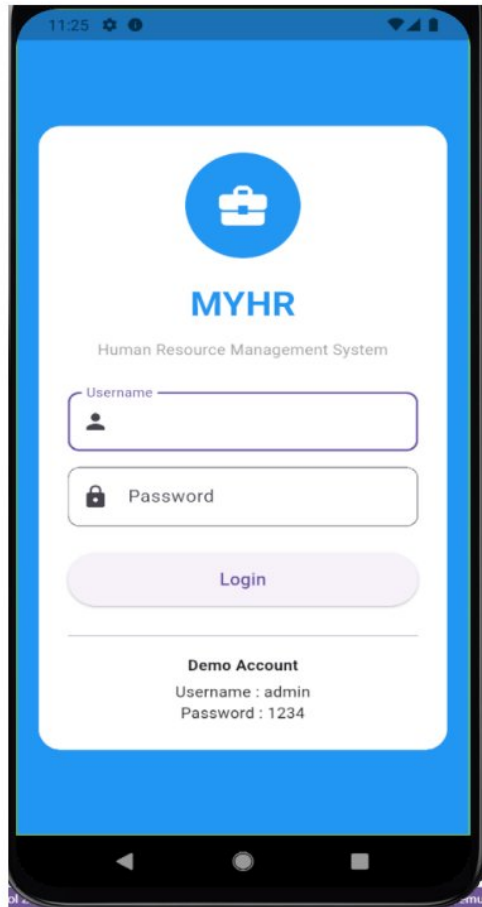
4. Tools & Technology

- VS code
- Dart Programming Language
- SQLite (sqflite package)
- Android Studio
- Emulator

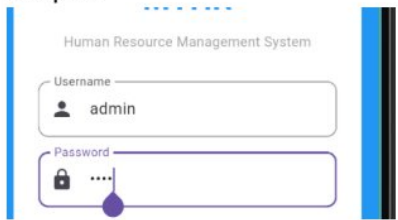
5. Application Interfaces

a) Main.dart (login page)

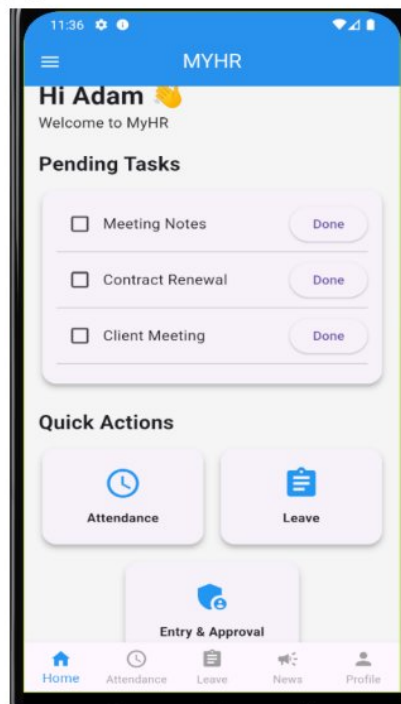
- Interface

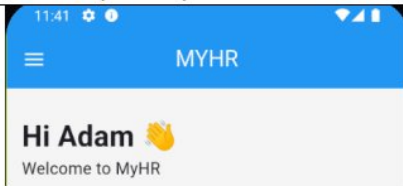


Functionalities	Proof
User must enter username and password ; Form validation to prevent empty fields.	A screenshot of the MYHR login interface, similar to the one above, but with a validation error. The 'Username' input field is empty, and a red error message 'Invalid Username or Password' is displayed at the bottom of the screen. The 'Password' field and 'Login' button are visible, but the 'Demo Account' section is not shown in this specific screenshot.

Password is hidden using obscureText	Coding: <pre> TextField(controller: passwordController, obscureText: true, decoration: InputDecoration(labelText: "Password", </pre> Output: 
After login, the user is redirected to the Home Page	- No relevant images

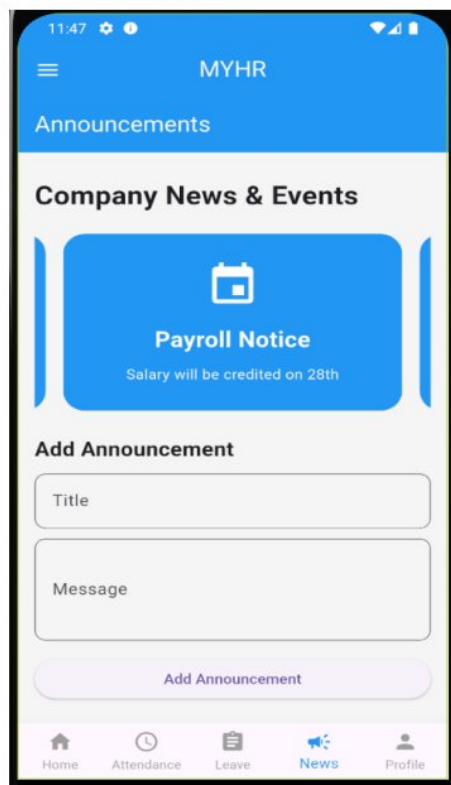
b) Home.dart (home page)

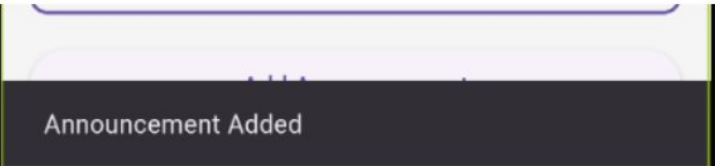


Functionalities	Proof
Acts as the dashboard	- The output of the homepage (above)
Contains a welcome section	

Includes quick navigation cards/buttons	Quick Actions Attendance Leave Entry & Approval
---	---

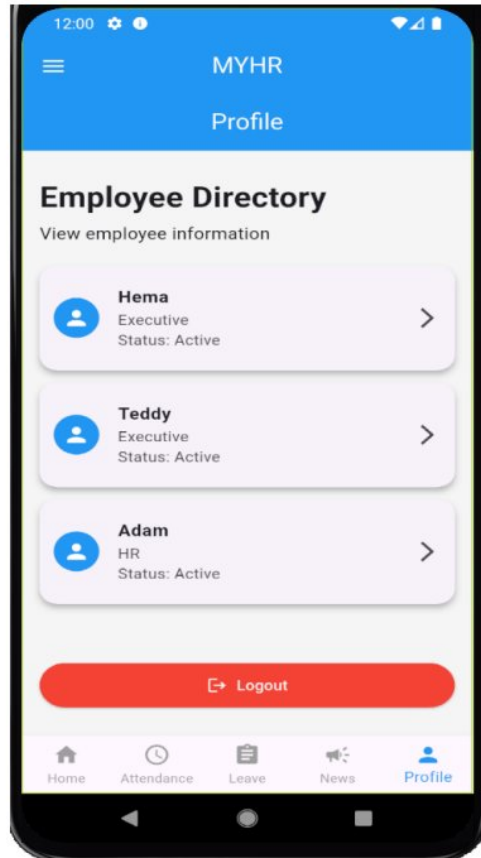
c) Announcement.dart (media page)

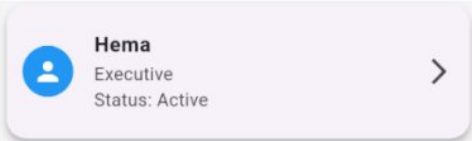
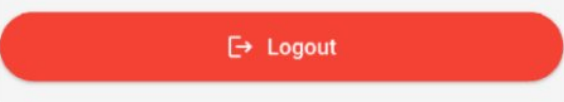


Functionalities	Proof
Displays company announcements	- The output of the announcement page (above)
Allows adding new announcements	

Displays list of announcements dynamically	<pre>// refresh carousel announcementRefresh.value++; if (!mounted) return; ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text("Announcement Added")),); }</pre>
Uses Carousel widget for featured announcements, displaying a minimum of 5 icons, titles and subtitles	Company News & Events  Building Event next month  Event Company E
Uses SQLite database for storage	<pre>import 'package:path/path.dart'; import 'package:sqflite/sqflite.dart'; // ===== // ANNOUNCEMENT // ===== Future<int> insertAnnouncement(String title, String message,) async { final db = await instance.database; return await db.insert('announcement', { 'title': title, 'message': message, },); }</pre>

d) Profile_page.dart



Functionalities	Proof
Displays the info of the users registered in the application	- The output of the profile page (above)
Displays user information using CircleAvatar	<pre>leading: const CircleAvatar(backgroundColor: Colors.blue, child: Icon(Icons.person, color: Colors.white,), // Icon), // CircleAvatar</pre> 
Includes Logout button	
Logout redirects to Login Page	<pre>void logout() { Navigator.pushAndRemoveUntil(context, MaterialPageRoute(builder: (context) => const LoginP (route) => false,); }</pre>

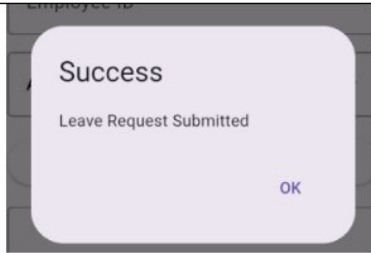
e) Leave_page.dart (formpage)

The screenshot shows a mobile application interface for 'MYHR'. The title is 'Leave Application'. The form contains the following elements:

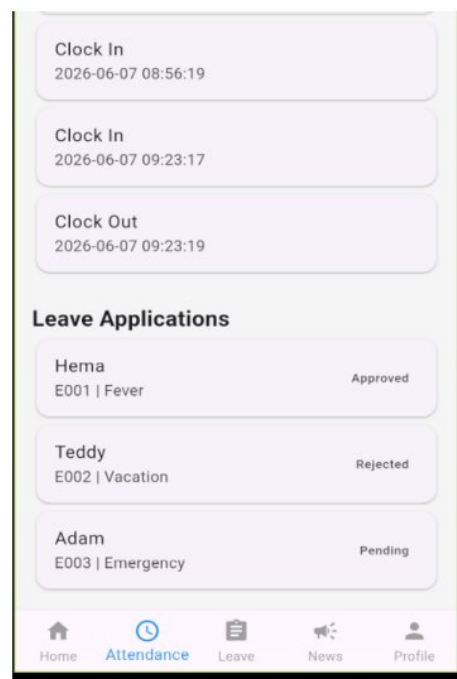
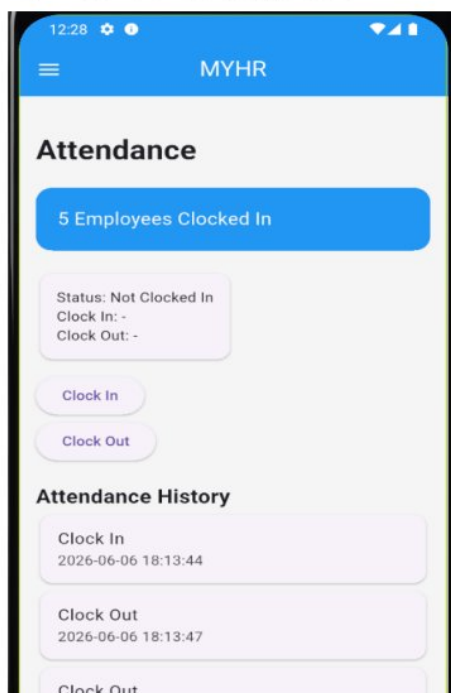
- Employee Name (text input)
- Employee ID (text input)
- Annual Leave (dropdown menu)
- Start Date (date picker)
- End Date (date picker)
- Reason (text area)
- Submit Leave (button)

 The bottom navigation bar has five icons: Home, Attendance, Leave (highlighted), News, and Profile.

Functionalities	Proof
Contains 3 input fields, 1 dropdown list, Calendar	The output of the leave page (above)
Uses validation to prevent empty input	<pre>), // InputDecoration validator: (value) => value!.isEmpty ? "Required" : null,), // TextFormField </pre> The close-up image shows the 'Employee Name' and 'Employee ID' input fields. Below each field, the word 'Required' is displayed in red text, indicating that the fields are mandatory and currently empty.

Shows AlertDialog on successful submission	
Uses TextFormField and controllers	<pre>TextFormField(controller: employeeIdController, decoration: const InputDecoration(labelText: "Employee ID", border: OutlineInputBorder(),), // InputDecoration</pre>

f) Attendance_page.dart



Functionality

The Attendance Page allows users to record their attendance activities. Users can perform Clock In and Clock Out actions, and the attendance records are stored in the SQLite database. The page also displays attendance information such as attendance status and recorded times.

Features

- Clock In functionality
- Clock Out functionality
- Attendance status display
- Stores attendance records in SQLite database
- Retrieves and displays attendance history

g) Entry_approval.dart (this page is to key in the info)

Data Entry & Approval

Add Data

Select Type
User

Employee Name

Employee ID

Department

Position

Email

Experience

Status

Add Employee

Data Entry & Approval

Add Data

Select Type
Task

Task Title

Add Task

Leave Approval

Hema
Employee ID : E001
Reason : Fever
Date : 2026-06-06 - 2026-06-07
Status : Approved
Approve Reject

Teddy
Employee ID : E002
Reason : Vacation
Date : 2026-06-08 - 2026-06-09
Approve Reject

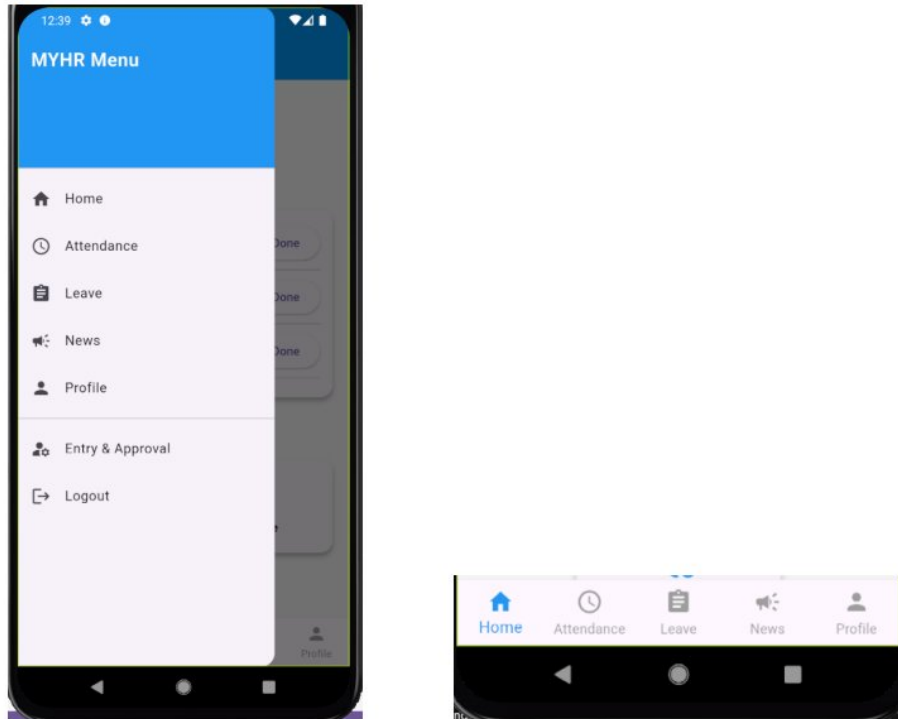
Functionality

The Entry Approval Page allows users to enter and submit information through input fields. The entered information is validated and stored in the database for future reference. This page serves as a form submission module within the application.

Features

- User input form
- Data validation
- Save information into the database
- Display confirmation message after submission
- Supports data management operations

h) Column.dart



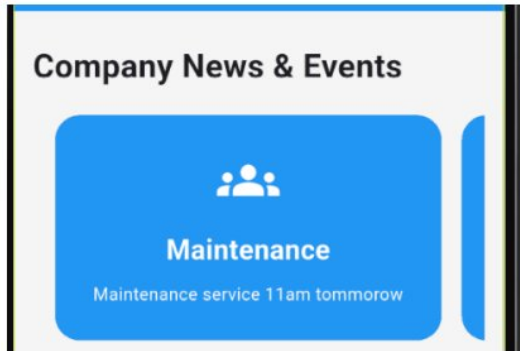
Functionality

The Column Page is the application's major navigation control. It uses Drawer Navigation and BottomNavigationBar to control page switching so that, from a single interface, users may reach several parts of the program.

Features

- BottomNavigationBar implementation
- Drawer Navigation implementation
- Navigation between Home, Media, Profile, and Form pages
- Centralized page management
- User-friendly navigation experience

i) Carousel_widget.dart



Functionality

The Carousel Widget displays announcements and important information in a sliding card format. Users can swipe horizontally to view multiple announcements. The widget improves the visual presentation of information within the application.

Features

- Horizontal sliding carousel
- Displays announcement title and message
- Uses PageView.builder
- Dynamic data loading from database
- Interactive and attractive user interface

j) Database_helper.dart

Functionality

The Database Helper class handles all of an application's database activities. For various modules like employees, attendance, announcements, leave requests, and activities, it is in charge of setting up database tables and managing CRUD (Create, Read, Update, Delete) operations.

Features

- SQLite database initialization
- Database table creation
- Insert data into the database
- Retrieve data from the database
- Update existing records
- Delete records
- Centralized database management

6. Database Implementation

Overview

The application uses **SQLite** as its local database management system through the **sqflit** package, the program uses SQLite as its local database management system. SQLite was chosen since it lets data be kept right on the device, free from outside server or internet connectivity demands.

Database Installation Procedure

Step 1: Set Up Dependencies

- To allow SQLite database capability, the required packages were added to the pubspec.yaml file.

```
! pubspec.yaml
1  name: hr_app2
2  description: "A new Flutter project."
3  # The following line prevents the package from being accidentally published to
4  # pub.dev using `flutter pub publish`. This is preferred for private packages.
5  publish_to: 'none' # Remove this line if you wish to publish to pub.dev
6  version: 1.0.0+1
7
8  environment:
9    sdk: ">=3.11.0 <4.0.0"
10
11  dependencies:
12    flutter:
13      sdk: flutter
14
15    cupertino_icons: ^1.0.8
16    sqflite: ^2.3.3
17    path: ^1.9.0
18
19  dev_dependencies:
20    flutter_test:
21      sdk: flutter
22
23    flutter_lints: ^6.0.0
24
25  flutter:
26    uses-material-design: true
27
```

- The SQLite database is created and maintained using sqflite.
- The path helps to create the right database file path on the device.

Step 2: Created Database Helper Class

All database-related activities were meant to be handled in a different file called `database_helper.dart`. This keeps the project in order and adheres to sound coding standards.

The `DatabaseHelper` class takes care of:

- Creating the database
- Creating table
- Inserting Data
- Retrieving data
- Updating data
- Deleting data

Step 3: Setup Database

When the program first runs, the database file `myhr.db` is generated.

```
_database = await _initDB('myhr.db');  
return _database!;  
}
```

Step 4: Create Database Tables

Several tables were created to store different types of application data.

Employee Table

Stores employee information.

Fields:

- Employee Name
- Employee ID
- Department
- Position
- Email
- Experience
- Status

Attendance Table

Stores attendance records.

Fields:

- Action (Clock In / Clock Out)
- Time

Leave Request Table

Stores leave application information.

Fields:

- Employee Name
- Employee ID
- Leave Type
- Reason
- Date
- Status

Announcement Table

Stores company announcements.

Fields:

- Title
- Message

Task Table

Stores task information.

Fields:

- Title
- Checked Status

Step 5 CRUD Operations Implemented

The application implements CRUD operations (Create, Read, Update, Delete).

Create (Insert Data)

Used when adding new records into the database.

Examples:

- Add employee
- Add announcement
- Add attendance record
- Add task
- Add leave request

Read (Retrieve Data)

Used to display information stored in the database.

Examples:

- View employee directory
- View announcements
- View attendance records
- View task list

Update (Modify Existing Data)

Used to update existing records.

Examples:

- Update leave request status
- Update task completion status

Delete (Remove Data)

Used to remove records from the database.

Examples:

- Delete task

Database Integration with Application Pages

Page	Database Usage
Profile Page	Retrieve employee information
Attendance Page	Store and display attendance records
Announcement Page	Store and display announcements
Entry Approval/Form Page	Store submitted information
Task Module	Store and update task records

7. Conclusion

This project clearly shows the creation of a Flutter mobile application, including numerous modules. It combines navigation, form validation, UI design, and database operations. Using the sqflite package, the Flutter app got the SQLite database fully incorporated. The database lets the program swiftly store and get information and supports all basic CRUD actions. The application improves understanding of real-world mobile app development concepts.